

Estruturas de Dados Básicas II

Exercícios de revisão

Daniel N. Pinheiro
daniel.pinheiro@dimap.ufrn.br



Implementação de pilha usando uma fila de máxima prioridade

Uma **pilha** (estrutura de dados do tipo **LIFO - Last-In-First-Out**) pode ser implementada utilizando uma **max-priority queue** internamente. Isso pode ser feito armazenando, em cada nó de uma **heap**, o par de valores "value" e "insertion_order", onde "value" é o valor do elemento e "insertion_order" é um número indicando a ordem em que ele foi inserido na pilha.

Por exemplo, se forem inseridos os elementos 10, 50 e 15 na pilha, a **heap** deve armazenar os pares (10, 1), (50, 2) e (15, 3), porque o valor 10 foi inserido primeiro, o valor 50 foi inserido segundo, e o valor 15 foi inserido terceiro.

Nesse sentido, podemos construir a seguinte estrutura para representar um nó de uma **heap**:

```
typedef struct{
    int value;
    int insertion_order;
} node;
```

Para que a **heap** leve em consideração a ordem em que os elementos forem inseridos, sua organização e posicionamento de nós deve considerar apenas os valores de "insertion_order". Dessa forma, o último elemento inserido na pilha será sempre o nó raiz da **max-heap**.

A implementação da `max_priority_queue` pode ser feita utilizando as seguintes variáveis:

```
class max_priority_queue {  
    int heap_size;  
    int insertion_count;  
    node A[MAX_LEN];  
};
```

A variável `insertion_count` deve indicar quantos elementos já foram inseridos na `max_priority_queue`, e ser incrementada a cada nova inserção. Note que o array `A` é do tipo `node`, para que possa armazenar tanto o valor do elemento quanto a ordem em que ele foi inserido.

Assim, as implementações das funções *push*, *pop*, e *top* de uma pilha podem ser feitas chamando as funções *insert*, *extract_max* e *get_max* de uma *max-priority queue*, respectivamente.

Complete as funções `max_heapify`, `get_max`, `extract_max` e `insert` do código abaixo para completar a implementação de uma pilha utilizando uma *max-priority queue*. Nestas funções, lembre-se de considerar quando utilizar os elementos `value` e `insertion_order` de um `node`.

O programa abaixo considera como entrada uma sequência de comandos do tipo "PUSH x", "POP" e "TOP", para inserir um elemento x, remover o elemento do topo da pilha (e retornar o valor removido), e retornar o topo da pilha, respectivamente. A leitura da entrada e a saída do programa já estão implementadas na função `main`.

Exemplo de entrada:

PUSH 100 PUSH 200 PUSH 300 POP PUSH 400 TOP POP TOP POP POP

Exemplo de saída:

300 400 400 200 200 100

```
37 void max_heapify(int i){
38     int largest;
39     int l = left(i);
40     int r = right(i);
41     if(l < this->heap_size && this->A[l].insertion_order > this->A[i].insertion_order){
42         largest = l;
43     } else {
44         largest = i;
45     }
46     if(r < this->heap_size && this->A[r].insertion_order > this->A[largest].insertion_order){
47         largest = r;
48     }
49     if(largest != i){
50         swap(this->A[i], this->A[largest]);
51         this->max_heapify(largest);
52     }
53 }
```

```
55 int get_max(){
56     return this->A[0].value;
57 }
```

```
59  int extract_max(){
60      int max = this->A[0].value;
61      this->A[0] = this->A[this->heap_size-1];
62      this->heap_size--;
63      this->max_heapify(0);
64      return max;
65  }
66
67  void insert(int v){
68      int i = this->heap_size++;
69      this->A[i].value = v;
70      this->A[i].insertion_order = ++this->insertion_count;
71      while(i > 0 && this->A[i].insertion_order > this->A[parent(i)].insertion_order){
72          swap(this->A[i], this->A[parent(i)]);
73          i = parent(i);
74      }
75  }
```

Exercícios

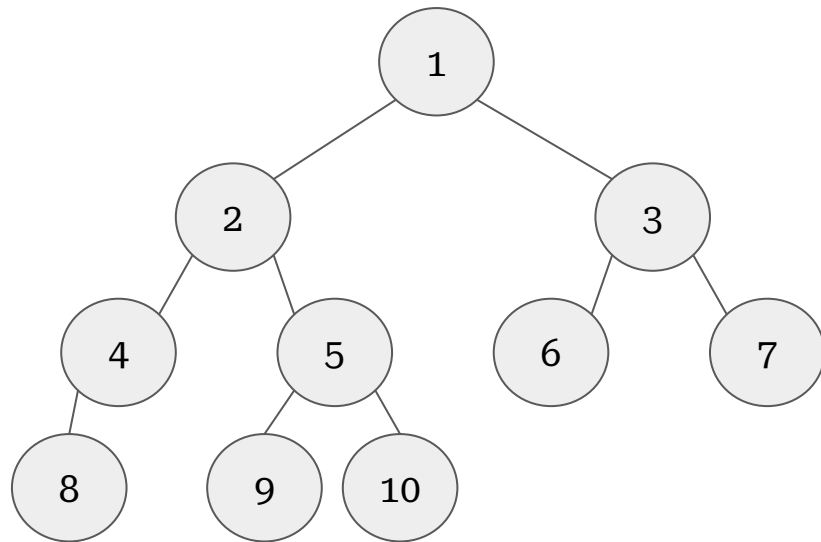
Considere o problema de busca de um elemento v em um array A de n elementos. Se o array estiver ordenado, então o algoritmo de busca pode testar o elemento do meio, comparando-o com v , e eliminar metade do array para continuar a busca recursivamente. O algoritmo de **busca binária** repete esse processo, diminuindo o subarray pela metade a cada iteração.

1. Escreva um algoritmo recursivo de tempo **$O(\lg n)$** para a busca binária, que deve retornar o índice de um elemento igual a v , ou o valor **-1** caso v não esteja no array.
2. Escreva uma **recorrência** para o tempo de execução de **pior caso** **$T(n)$** desse algoritmo.
3. Mostre pelo **método da substituição** que o tempo de execução de pior caso é **$\Theta(\lg n)$** .

Exercícios

Escreva a ordem em que os valores dos nós da árvore ao lado seriam impressos se fossem utilizadas as seguintes ordens de travessia:

1. *Pre-order.*
2. *In-order.*
3. *Post-order.*



Exercícios

Expresse a altura h de uma árvore binária de n nós, sem conhecer sua estrutura, utilizando as notações O e Ω . Ou seja, defina $f(n)$ e $g(n)$, de modo que $h = \Omega(f(n))$ e $h = O(g(n))$.

Principais operações em *heaps*

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

Max-Heapify assume que as subárvores com raiz em **Left(i)** e **Right(i)** são Max-Heaps, mas **$A[i]$** pode ser menor que seus filhos, violando a propriedade da Max-Heap.

Principais operações em *heaps*

BUILD-MAX-HEAP(A, n)

```
1   $A.heap-size = n$   
2  for  $i = \lfloor n/2 \rfloor$  downto 1  
3      MAX-HEAPIFY( $A, i$ )
```

Build-Max-Heap converte um *array* $A[1 : n]$ chamando Max-Heapify de baixo pra cima.

Principais operações em *heaps*

HEAPSORT(A, n)

1 BUILD-MAX-HEAP(A, n)

2 **for** $i = n$ **downto** 2

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)

Principais operações em *heaps*

- Busca do valor máximo

MAX-HEAP-MAXIMUM(A)

```
1  if  $A.heap-size < 1$   
2      error “heap underflow”  
3  return  $A[1]$ 
```

- Extração do valor máximo

MAX-HEAP-EXTRACT-MAX(A)

```
1   $max = \text{MAX-HEAP-MAXIMUM}(A)$   
2   $A[1] = A[A.heap-size]$   
3   $A.heap-size = A.heap-size - 1$   
4   $\text{MAX-HEAPIFY}(A, 1)$   
5  return  $max$ 
```

Principais operações em *heaps*

- Incremento da chave de um elemento x

MAX-HEAP-INCREASE-KEY(A, x, k)

```
1  if  $k < x.key$ 
2      error “new key is smaller than current key”
3   $x.key = k$ 
4  find the index  $i$  in array  $A$  where object  $x$  occurs
5  while  $i > 1$  and  $A[\text{PARENT}(i)].key < A[i].key$ 
6      exchange  $A[i]$  with  $A[\text{PARENT}(i)]$ , updating the information that maps
        priority queue objects to array indices
7       $i = \text{PARENT}(i)$ 
```

Principais operações em *heaps*

- Inserção de um novo elemento x

MAX-HEAP-INSERT(A, x, n)

```
1  if  $A.heap-size == n$   
2      error “heap overflow”  
3   $A.heap-size = A.heap-size + 1$   
4   $k = x.key$   
5   $x.key = -\infty$   
6   $A[A.heap-size] = x$   
7  map  $x$  to index  $heap-size$  in the array  
8  MAX-HEAP-INCREASE-KEY( $A, x, k$ )
```

Bibliografia

- CORMEN, Thomas H. et al. **Introduction to algorithms**. 3rd ed. Cambridge: MIT, c2009. xix, 1292 p. ISBN: 9780262033848.

